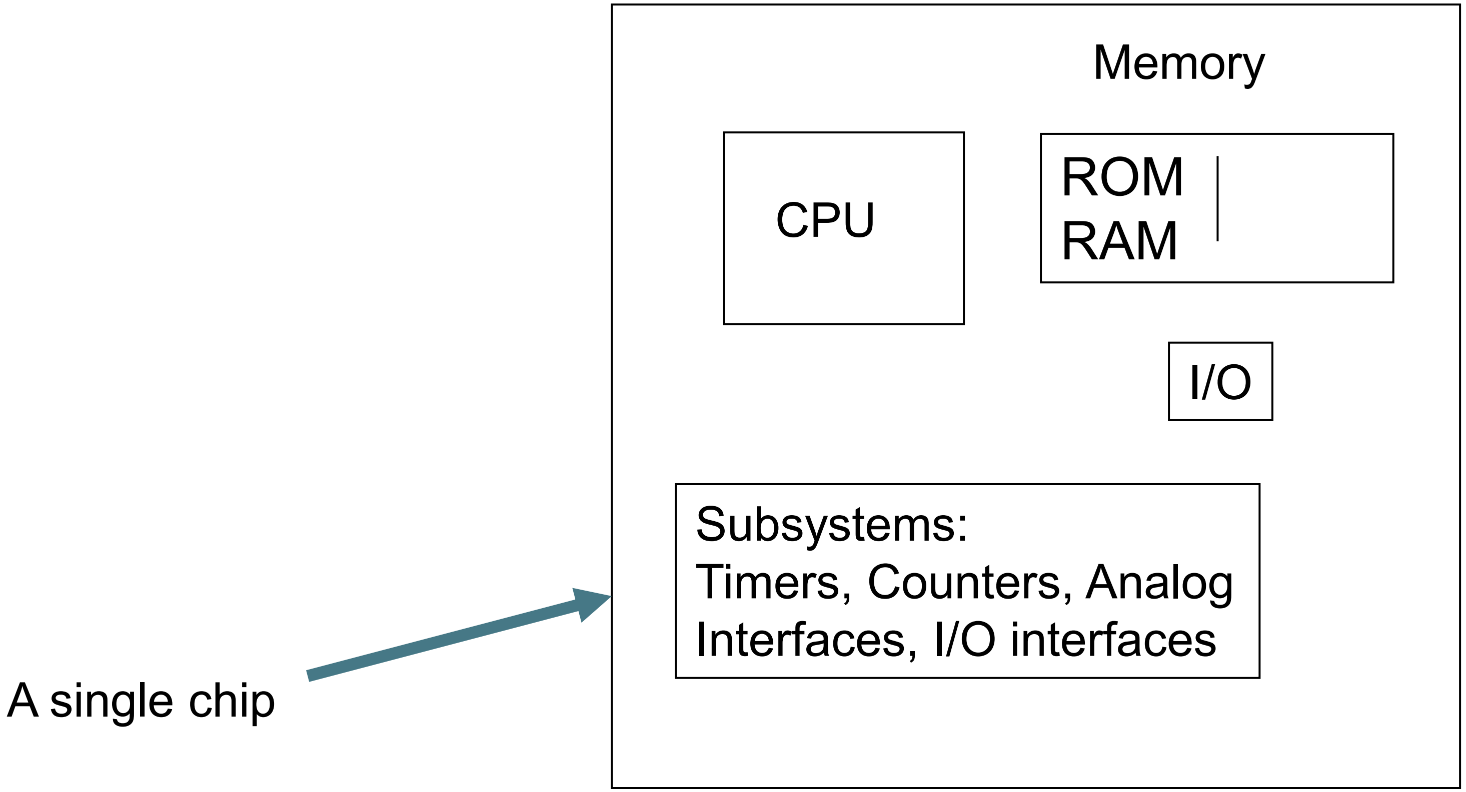
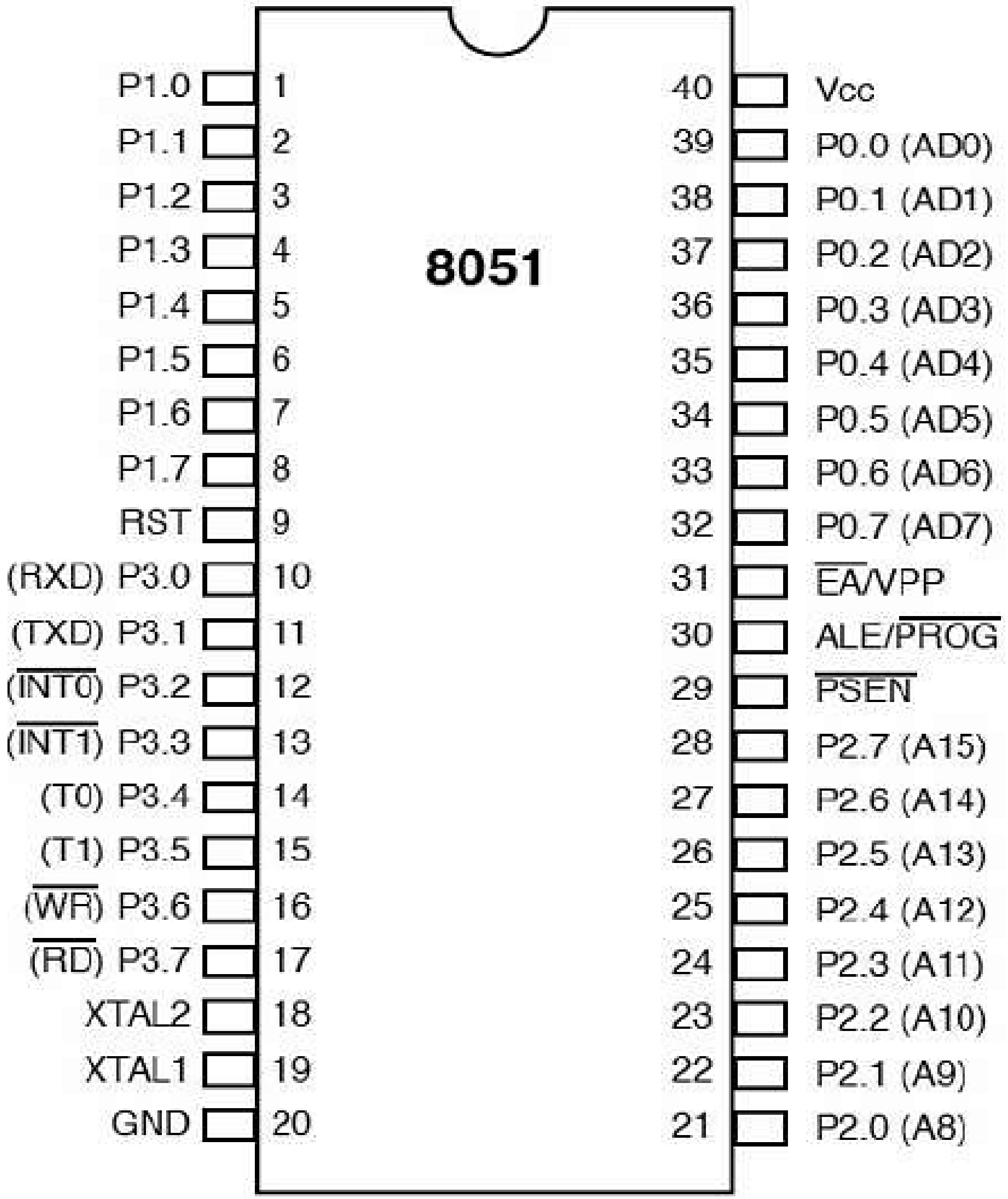
IoT Prototyping Laboratory

Suchismita Chinara

Microcontrollers

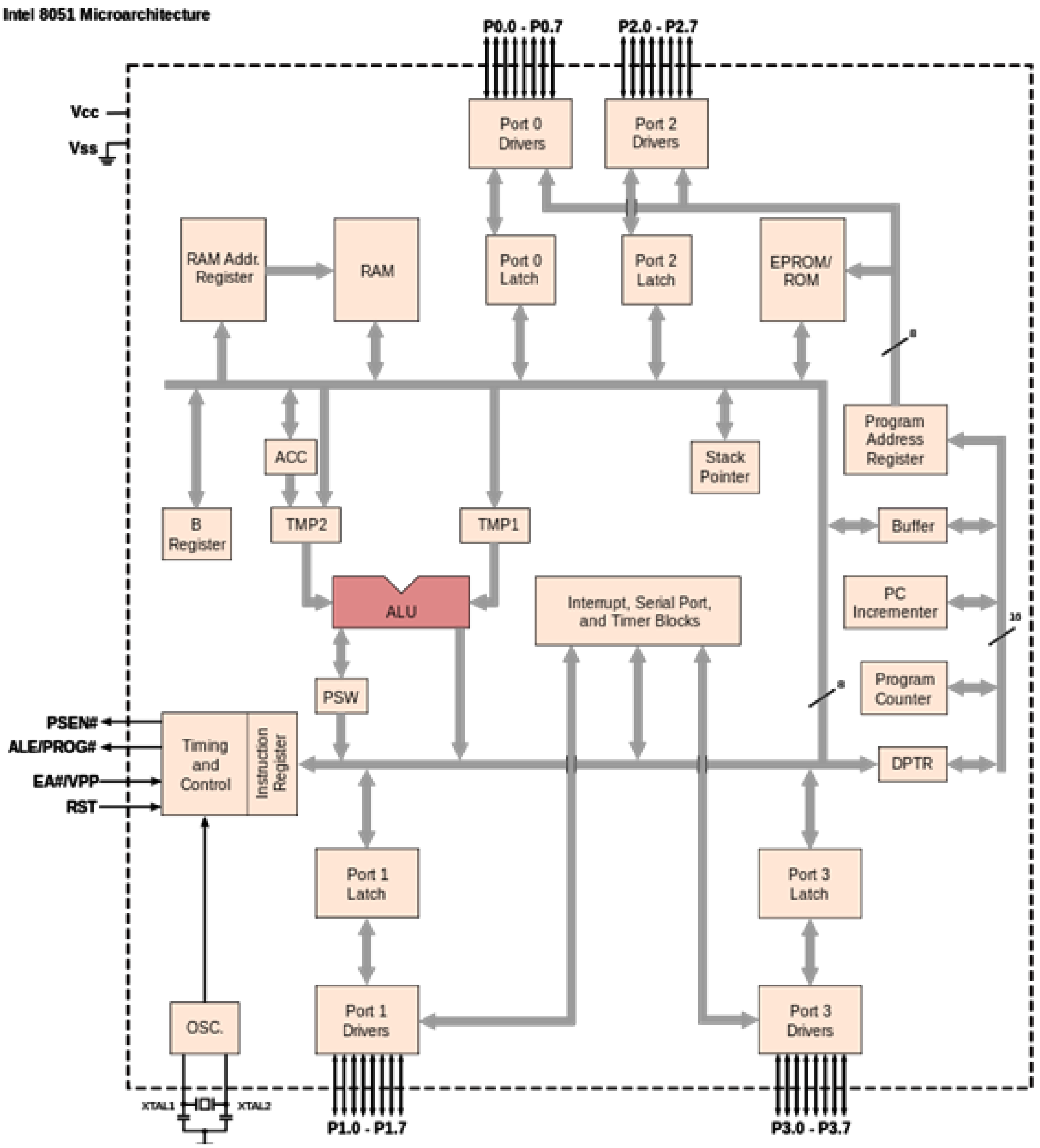


Pin Diagram



8051
P1.0 1
P1.1 2
P1.2 3
P1.3 4
P1.4 5
P1.5 6
P1.6 7
P1.7 8
RST 9
(RXD) P3.0 10
(TXD) P3.1 11
($\overline{INT0}$) P3.2 12
($\overline{INT1}$) P3.3 13
(T0) P3.4 14
(T1) P3.5 15
($\overline{WR}$) P3.6 16
($\overline{RD}$) P3.7 17
XTAL2 18
XTAL1 19
GND 20
40 Vcc
39 P0.0 (AD0)
38 P0.1 (AD1)
37 P0.2 (AD2)
36 P0.3 (AD3)
35 P0.4 (AD4)
34 P0.5 (AD5)
33 P0.6 (AD6)
32 P0.7 (AD7)
31 $\overline{EA}$/VPP
30 ALE/$\overline{PROG}$
29 $\overline{PSEN}$
28 P2.7 (A15)
27 P2.6 (A14)
26 P2.5 (A13)
25 P2.4 (A12)
24 P2.3 (A11)
23 P2.2 (A10)
22 P2.1 (A9)
21 P2.0 (A8)

Architecture



Ports

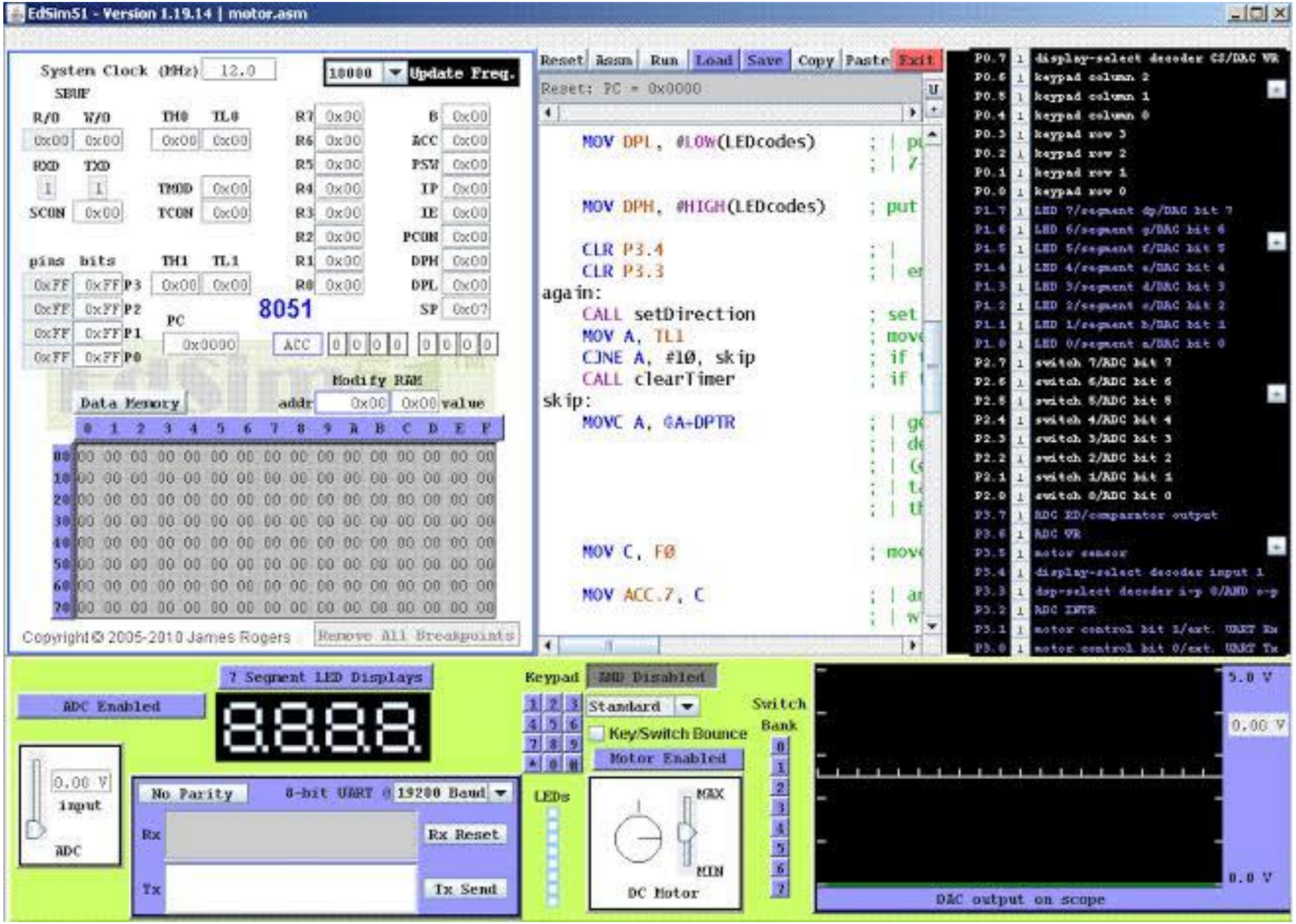
- **PORT P1 (Pins 1 to 8)**: The port P1 is a port dedicated for general I/O purpose. The other ports P0, P2 and P3 have dual roles in addition to their basic I/O function.
- **PORT P0 (pins 32 to 39):** When the external memory access is required then Port P0 is multiplexed for address bus and data bus that can be used to access external memory in conjunction with port P2. P0 acts as A0-A7 in address bus and D0-D7 for port data. It can be used for general purpose I/O if no external memory presents.
- **PORT P2 (pins 21 to 28)**: Similar to P0, the port P2 can also play a role (A8-A15) in the address bus in conjunction with PORT P0 to access external memory.

Contd.

- **PORT P3 (Pins 10 to 17)**:
  - In addition to acting as a normal I/O port,
    - P3.0 can be used for serial receive input pin (RXD)
    - P3.1 can be used for serial transmit output pin (TXD) in a serial port,
    - P3.2 and P3.3 can be used as external interrupt pins (INT0’ and INT1’),
    - P3.4 and P3.5 are used for external counter input pins (T0 and T1),
    - P3.6 and P3.7 can be used as external data memory write and read control signal pins (WR’ and RD’) read and write pins for memory access.

Installation

- Navigate to https://sourceforge.net/projects/edsim51/
- Download the file.
- Install Java (Download x64 for Windows and can choose for Mac).
- https://www.java.com/en/download/manual.jsp
- If you are on Linux (Ubuntu), use
  - sudo apt install default-jdk
  - Install Java.
- Unzip the EdSim51 File and open the .jar file.


EdSim51 - Version 1.19.14 | motor.asm
System Clock (MHz) 12.0
10000 Update Freq.
Reset Assm Run Load Save Copy Paste Exit
Reset: PC = 0x0000
MOV DPL, #LOW(LEDcodes)
MOV DPH, #HIGH(LEDcodes)
CLR P3.4
CLR P3.3
again:
CALL setDirection
MOV A, TL1
CJNE A, #10, skip
CALL clearTimer
skip:
MOVC A, @A+DPTR
MOV C, F0
MOV ACC.7, C
8051
Data Memory
Modify RAM
Copyright © 2005-2010 James Rogers
Remove All Breakpoints
7 Segment LED Displays
ADC Enabled
Keypad
Standard
Key/Switch Bounce
Motor Enabled
Switch Bank
No Parity
8-bit UART @ 19200 Baud
Rx Reset
Tx Send
LEDs
DC Motor
DAC output on scope
5.0 V
0.0 V

First Code

1. Org 00H ; Start of Program
2. Start: Mov A, #0FH ; This is comment. Move copies the value
3. Mov P1, A
4. End

Org: Is the Origin of the program. It tells us where in the memory the program starts.

End: Is used to inform that the program is ending.

Instructions for 8051

[label:] mnemonic [operands] [; comments]

Example: Start: Mov A,#0FH ; This copy data into A

Label Instructions Comments

Opcode Operand

Mov A,#0FH

The First code in memory

- This link contains 8051 instructions by HEX code.

https://www.keil.com/support/man/docs/is51/is51_opcodes.asp?bhcp=1

- 74 0F
- F5 90

- The above values are the hex values in the data memory for the first code.

Second Code

```
1.            Org 00H
2. Start:   Mov A, #05H
3.          Add A, #010H ; Add the value in the accumulator
4.          End
```

- This is the Accumulator adding two values

DIY

- Do the multiplication of 2 numbers.

Multiply two values

- Org 00H
- Start: Mov A, #03H
- Mov B, #02H
- mul ab ; A and B are multiplied
- End

Immediate Data movement

1. Org 00H
2. Start: ; Immediate Data (Constants) - REQUIRES '#'
3. MOV A, #45H ; Load Hex value 45 into Accumulator
4. MOV R0, #10 ; Load Decimal value 10 into Register R0
5. MOV DPTR, #1234H ; Load 16-bit address into Data Pointer
6. End

Direct Addressing

1. Org 00H
2. Start: ; Direct Addressing (RAM / Registers / Ports) - NO '#'
3. 0000| MOV A, P2 ; Copy 8-bit input from Port 2 into Accumulator
4. 0002| MOV P1, A ; Send Accumulator bits out to Port 1 LEDs
5. 0004| MOV R1, 30H ; Copy content of RAM location 30H into R1
6. End

Indirect Addressing

1. Org 00H
2. Start: ; Indirect Addressing (Pointers) - USES '@'
3. 0000| MOV A, @R0 ; Read RAM byte whose address is stored inside R0
4. 0001| MOV @R1, A ; Write Accumulator byte into RAM address stored in R1
5. End

Number System

- MOV A, #0FFH ; Hexadecimal (Must start with a digit 0-9 if starting with A-F)
- MOV A, #11001010B ; Binary (suffix 'B')
- MOV A, #255 ; Decimal (no suffix)
- MOV A, #'A' ; ASCII character (evaluates to 41H / 65)

Binary Pattern in Port 1 LEDs

```
1.      org 00h
2.  start:
3.      DEC P1   ;Decrease
4.      JMP start ; Jump back to the start variable
5.  end
```

Contd.

- **DEC P1**
  - **DEC (Decrement):** Subtracts 1 from the 8-bit value inside **Port 1 (P1)**.
  - On reset, all port pins default to high/1s (FFH or 11111111B).
    - **1st run:** P1 drops from 11111111B (FFH) to 11111110B (FEH).
    - **2nd run:** Drops to 11111101B (FDH).
    - **3rd run:** Drops to 11111100B (FCH), and continues counting down to 00000000B before rolling back over to 11111111B.
- **JMP start**
  - **JMP (Unconditional Jump):** Forces the 8051 to jump back to the **start:** label, forming an infinite loop that keeps decrementing P1 forever.

Switching LED's on Click

1. org 00h
2. start:
3. MOV P1, P2 ; move data on P2 pins to P1
4. JMP start ; and repeat
5. end

Assignment

1. Do the subtraction of 2 numbers.
2. Do the division of 2 numbers.
3. Do addition with carry.
4. Do subtraction with borrow.